

Danny Topete, dtpe004, CS166 - Spring 2026, April 17

University Database Screenshot

Figure 1: Running university.sql

University SQL code

```
-- ***** University Database *****
```

```
-- Defining entities
```

```
CREATE TABLE Professor(ssn CHAR(11) NOT NULL, name CHAR(30), age INTEGER, speciality  
CHAR(30), rank CHAR(20), PRIMARY KEY(ssn));
```

```
CREATE TABLE Dept(dno INTEGER NOT NULL, dname CHAR(30), office CHAR(30), PRIMARY  
KEY(dno));
```

```
CREATE TABLE Project(pno INTEGER NOT NULL, sponsor CHAR(30), start_date DATE,  
end_date DATE, budget INTEGER, PRIMARY KEY(pno));
```

```
CREATE TABLE Graduate(ssn CHAR(11) NOT NULL, name CHAR(30), age INTEGER, deg_pq  
CHAR(30), PRIMARY KEY(ssn));
```

```
-- Defining relations
```

```
-- Senior Graduate student advises a grad student
```

```
CREATE TABLE Advise(senior_ssn CHAR(11) NOT NULL, grad_ssn CHAR(11) NOT NULL, PRIMARY  
KEY(grad_ssn), FOREIGN KEY(senior_ssn) REFERENCES Graduate(ssn), FOREIGN  
KEY(grad_ssn) REFERENCES Graduate(ssn), CHECK(senior_ssn <> grad_ssn));
```

```
-- Graduate student has a major (relationship) between Graduate and Dept
```

```
CREATE TABLE Major(grad_ssn CHAR(11) NOT NULL, dno INTEGER NOT NULL, PRIMARY  
KEY(grad_ssn), FOREIGN KEY(grad_ssn) REFERENCES Graduate(ssn), FOREIGN KEY(dno)  
REFERENCES Dept(dno));
```

```
-- Dept are ran by one professor
```

```
CREATE TABLE runs(dno INTEGER NOT NULL, ssn CHAR(11) NOT NULL, PRIMARY KEY(dno),  
FOREIGN KEY(dno) REFERENCES Dept(dno), FOREIGN KEY(ssn) REFERENCES Professor(ssn));
```

```
-- Every professor works for a work_dept (relationship) between Professor and Dept
```

```
CREATE TABLE work_dept(time_pm INTEGER, ssn CHAR(11) NOT NULL, dno INTEGER NOT NULL,  
PRIMARY KEY(ssn, dno), FOREIGN KEY(ssn) REFERENCES Professor(ssn), FOREIGN KEY(dno)  
REFERENCES Dept(dno));
```

```
-- Every project in manage(relationship) between Professor and Project
```

```
CREATE TABLE manage(ssn CHAR(11) NOT NULL, pno INTEGER NOT NULL, PRIMARY KEY(ssn,  
pno), FOREIGN KEY(ssn) REFERENCES Professor(ssn), FOREIGN KEY(pno) REFERENCES  
Project(pno));
```

```
-- Every project has a professor that supervises it (relationship) between Professor  
and Project
```

```
CREATE TABLE supervise(ssn CHAR(11) NOT NULL, pno INTEGER NOT NULL, PRIMARY KEY(ssn,  
pno), FOREIGN KEY(ssn) REFERENCES Professor(ssn), FOREIGN KEY(pno) REFERENCES  
Project(pno));
```

```
-- Every project can have many professors working on it
```

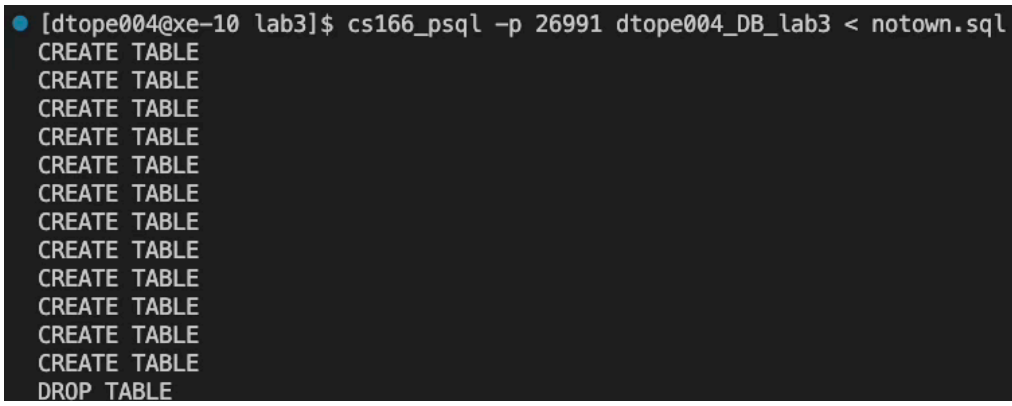
```
CREATE TABLE work_in(ssn CHAR(11) NOT NULL, pno INTEGER NOT NULL, PRIMARY KEY(ssn,  
pno), FOREIGN KEY(ssn) REFERENCES Professor(ssn), FOREIGN KEY(pno) REFERENCES  
Project(pno));
```

```
-- work_proj(relationship) between Professor, Graduate, and Project with the  
attribute since
```

```
CREATE TABLE work_proj(ssn CHAR(11) NOT NULL, grad_ssn CHAR(11) NOT NULL, pno INTEGER  
NOT NULL, since DATE, PRIMARY KEY(grad_ssn, pno), FOREIGN KEY(ssn) REFERENCES  
Professor(ssn), FOREIGN KEY(grad_ssn) REFERENCES Graduate(ssn), FOREIGN KEY(pno)  
REFERENCES Project(pno));
```

```
DROP TABLE IF EXISTS Advise, Major, runs, work_dept, manage, supervise, work_in,  
work_proj;
```

Notown Screenshot

A terminal window with a dark background and light blue text. The prompt is [dtape004@xe-10 lab3]\$ and the command is cs166_psql -p 26991 dtape004_DB_lab3 < notown.sql. The output consists of 14 lines of SQL commands: 13 'CREATE TABLE' statements followed by 1 'DROP TABLE' statement.

```
● [dtape004@xe-10 lab3]$ cs166_psql -p 26991 dtape004_DB_lab3 < notown.sql
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
DROP TABLE
```

Figure 2: Running notown.sql

Notown SQL code

```
-- ***** Notown Records Database *****
CREATE TABLE Instrument(instrid INTEGER NOT NULL, dname CHAR(30), key CHAR(10),
PRIMARY KEY(instrid));
CREATE TABLE Musicians(ssn CHAR(11) NOT NULL, name CHAR(30), PRIMARY KEY(ssn));
-- A musician plays an instrument (relationship) between Musicians and Instrument
CREATE TABLE Plays(ssn CHAR(11) NOT NULL, instrid INTEGER NOT NULL, PRIMARY KEY(ssn,
instrid), FOREIGN KEY(ssn) REFERENCES Musicians(ssn), FOREIGN KEY(instrid) REFERENCES
Instrument(instrid));

-- Each song appears in album
CREATE TABLE Songs(songid INTEGER NOT NULL, title CHAR(30), author CHAR(30), PRIMARY
KEY(songid));
CREATE TABLE Album(albumid INTEGER NOT NULL, copyrightDate DATE, speed INTEGER, title
CHAR(30), PRIMARY KEY(albumid));
CREATE TABLE Appears(songid INTEGER NOT NULL, albumid INTEGER NOT NULL, PRIMARY
KEY(songid), FOREIGN KEY(songid) REFERENCES Songs(songid), FOREIGN KEY(albumid)
REFERENCES Album(albumid));

-- Albums are Produced by a musician (relationship) between Album and Musicians
CREATE TABLE Producer(albumid INTEGER NOT NULL, ssn CHAR(11) NOT NULL, PRIMARY
KEY(albumid), FOREIGN KEY(albumid) REFERENCES Album(albumid), FOREIGN KEY(ssn)
REFERENCES Musicians(ssn));

-- Songs are performed by Musicians
CREATE TABLE Perform(songid INTEGER NOT NULL, ssn CHAR(11) NOT NULL, PRIMARY
KEY(songid, ssn), FOREIGN KEY(songid) REFERENCES Songs(songid), FOREIGN KEY(ssn)
REFERENCES Musicians(ssn));

-- Each musician that records at Notown has an SSN, a name, an address and a phone
number. Poorly paid musicians often share the same address, and no address has more
than one phone.
-- Home is a relationship between the entity Place and Telephone.
CREATE TABLE Place(address CHAR(30) NOT NULL, PRIMARY KEY(address));
CREATE TABLE Telephone(phone_no CHAR(15) NOT NULL, PRIMARY KEY(phone_no));
CREATE TABLE Home(address CHAR(30) NOT NULL, phone_no CHAR(15) NOT NULL, PRIMARY
KEY(address), FOREIGN KEY(address) REFERENCES Place(address), FOREIGN KEY(phone_no)
REFERENCES Telephone(phone_no));
-- Lives is a relationship between Musicians and Place
CREATE TABLE Lives(ssn CHAR(11) NOT NULL, address CHAR(30) NOT NULL, PRIMARY
KEY(ssn), FOREIGN KEY(ssn) REFERENCES Musicians(ssn), FOREIGN KEY(address) REFERENCES
Place(address));

DROP TABLE IF EXISTS Plays, Appears, Producer, Perform, Home, Lives;
```